

中山大學
SUN YAT-SEN UNIVERSITY

并行硬件和并行软件 (Chap 2.1, 2.2)

王桢

wangzh665@mail.sysu.edu.cn

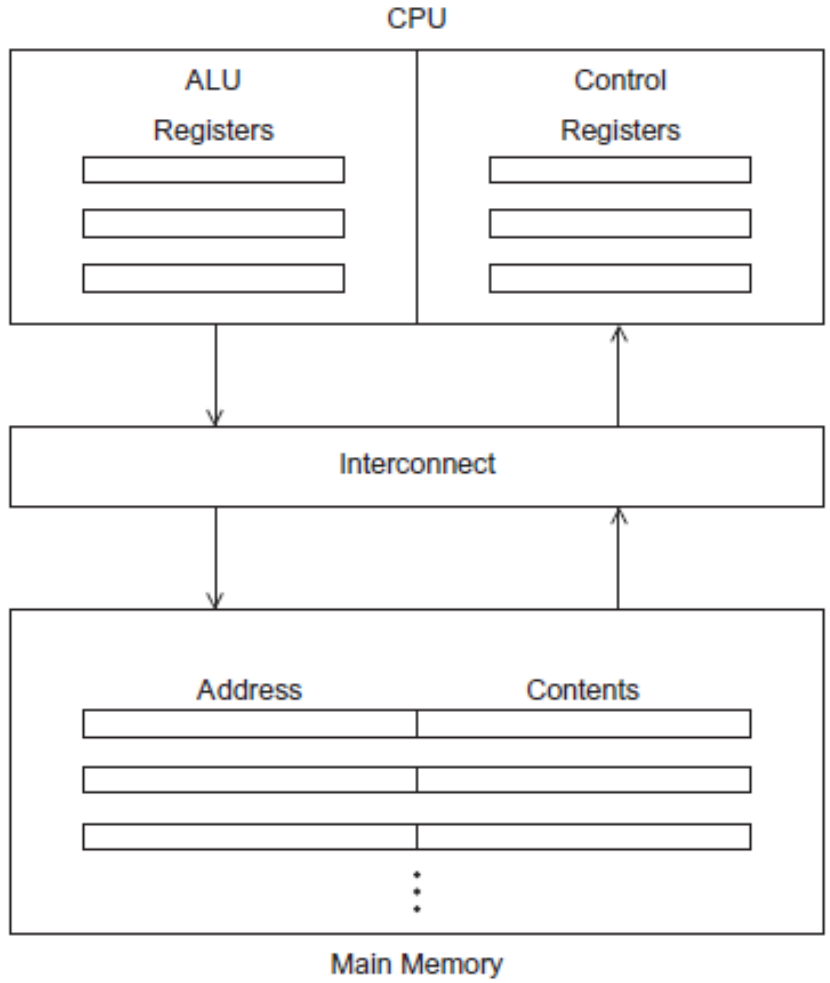
目录

- The von Neumann architecture
  - 内存和处理器的分离是瓶颈所在
- Processes, multitasking, and threads
- Caching
- Virtual memory
- Instruction-level parallelism
- Hardwear multithreading

Modifications to the von Neumann model

The von Neumann architecture

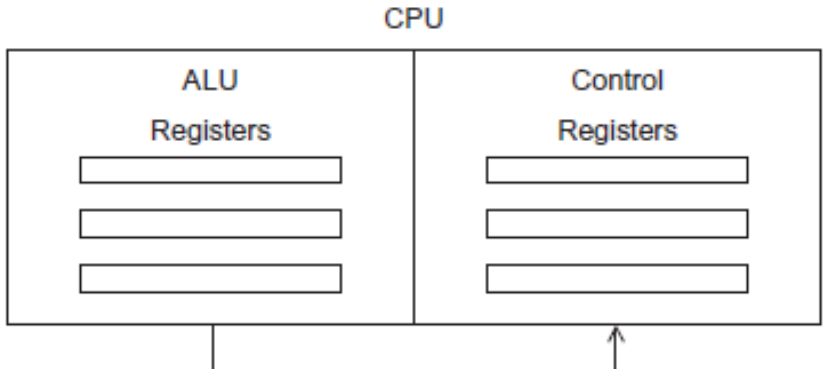
- 处理器 (CPU/core/processor)通过总线 (interconnect/bus)读写内存 (main memory)
- Why this separation?
  - compute vs storage want different “physics”
- Advantages
  - Cost & capacity
  - Modularity & flexibility
  - Scalability across many cores / devices



CPU

- Divided into two parts
  - **Control unit** (*the boss*) - responsible for deciding which instruction in a program should be executed.
  - **Arithmetic and logic unit** (ALU) (*the worker*) - responsible for executing the actual instructions.
- Key terms
  - **Register** – very fast storage, part of the CPU.
  - **Program counter** – stores address of the next instruction to be executed.

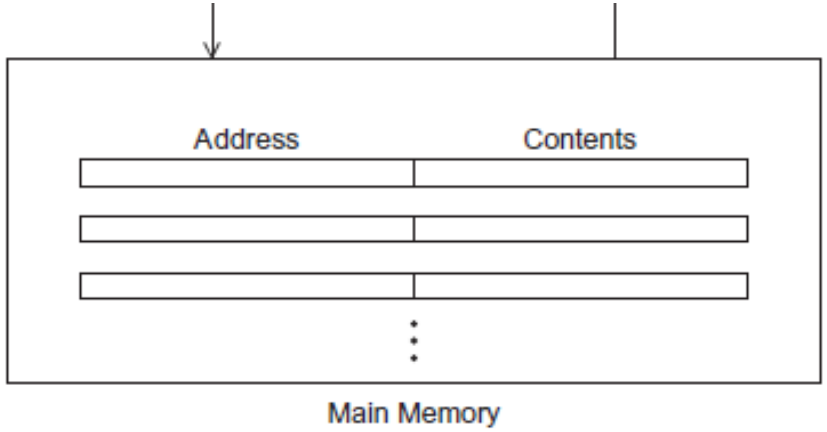


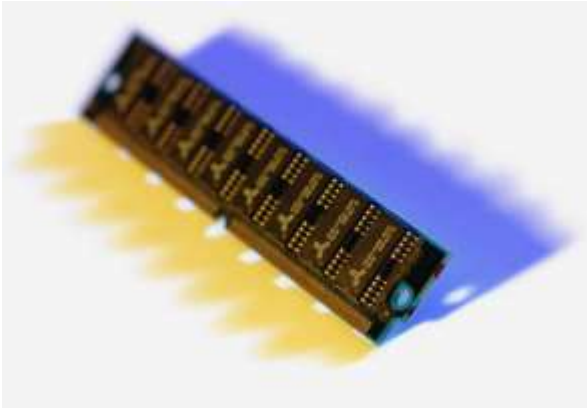


Main memory

- This is a collection of locations, each of which is capable of storing both instructions and data.

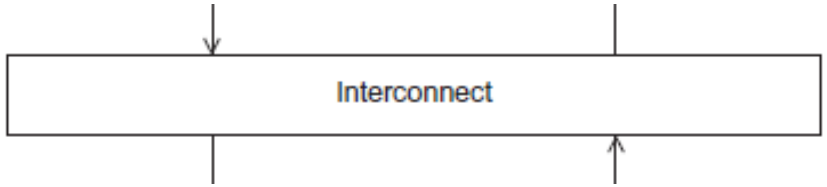
- Every location consists of an address, which is used to access the location, and the contents of the location.





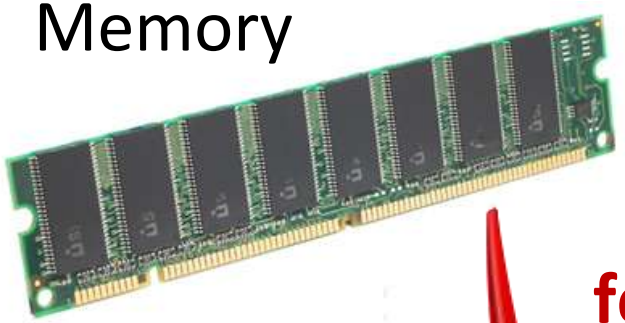
Interconnect

- Wires and hardware that connects the CPU and memory
- Carries Address + Data + Control signals
- Key performance metrics
  - Bandwidth and Latency
- Shared vs. switched fabric
  - Crucial for multi-core / processor scenarios

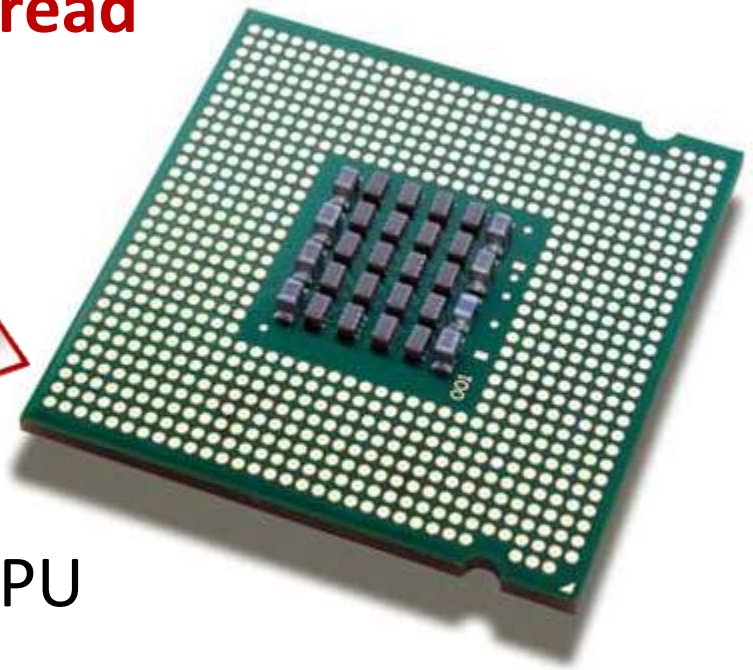




Memory

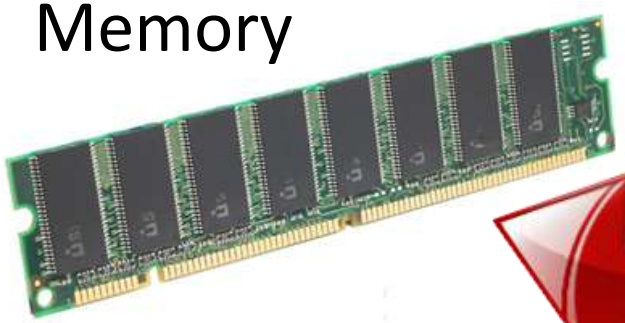


fetch/read



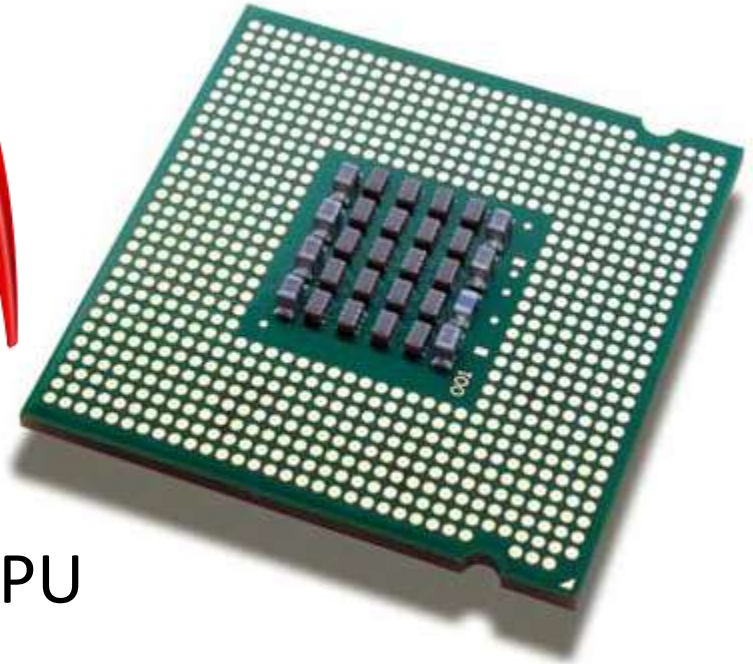
CPU

Memory



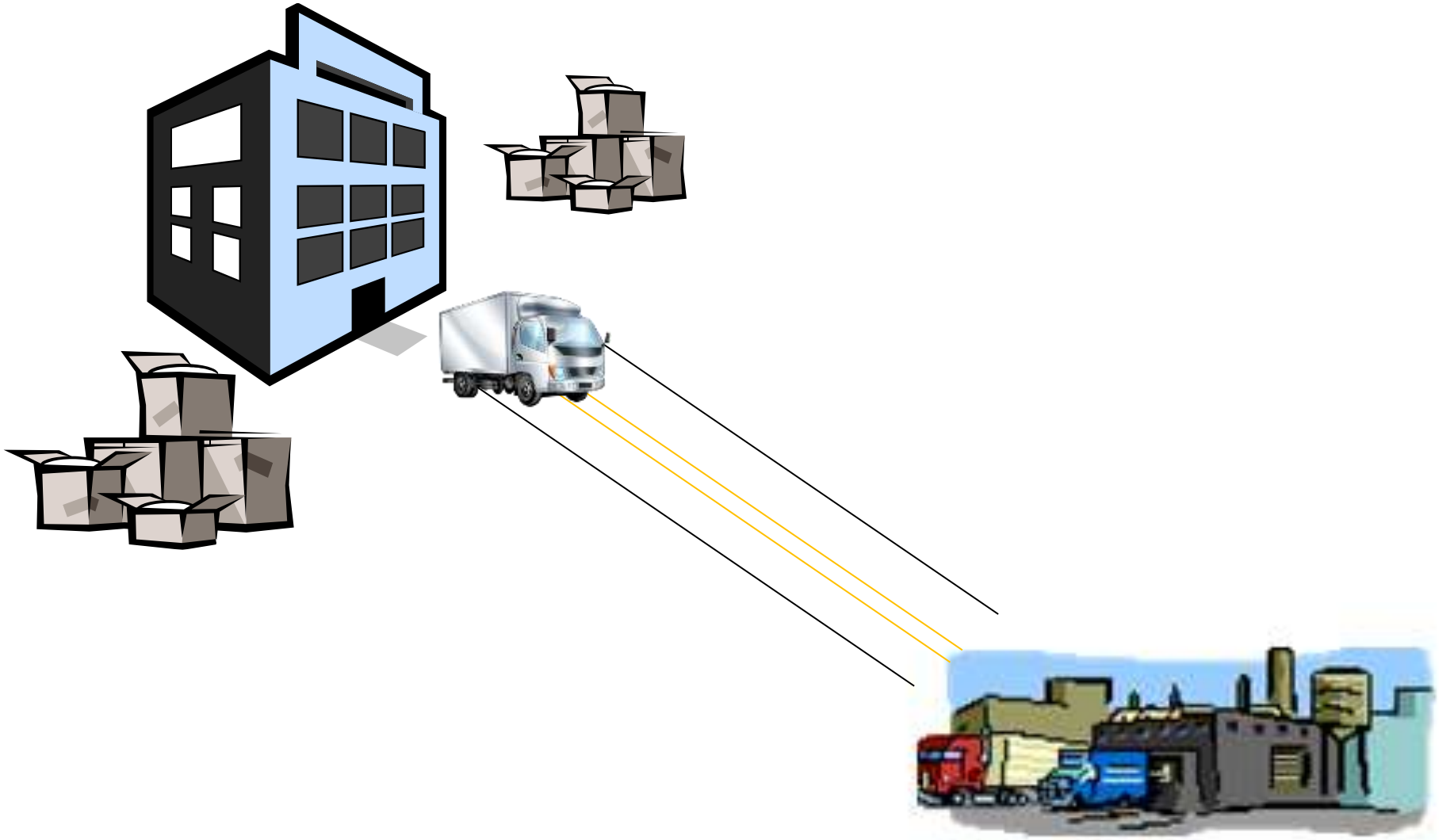


write/store



CPU

von Neumann bottleneck



目录

- The von Neumann architecture
- Processes, multitasking, and threads
  - 理解操作系统的基本机制
- Caching
- Virtual memory
- Instruction-level parallelism
- Hardwear multithreading

Modifications to the von Neumann model

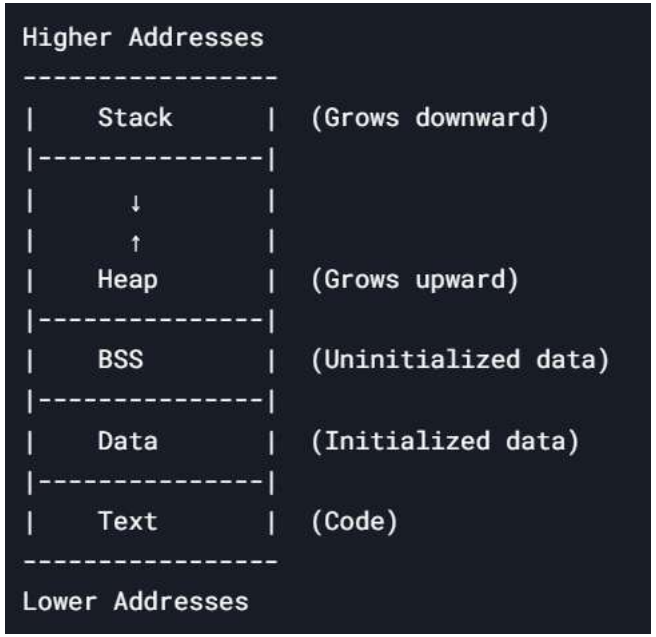
An operating system “process”



中山大學
SUN YAT-SEN UNIVERSITY

- An instance of a computer program that is being executed.
- Components of a process:
  - The executable machine language program.
  - Address space. Different processes have **separate address spaces** (isolation)
  - Descriptors of resources the OS has allocated to the process.
  - Security information.
  - Information about the state of the process.

Typical virtual memory layout for a process



Threading



- Threads are contained within processes.
- They allow programmers to divide their programs into (more or less) independent tasks.
- The hope is that when one thread blocks because it is waiting on a resource, another will have work to do and can run.

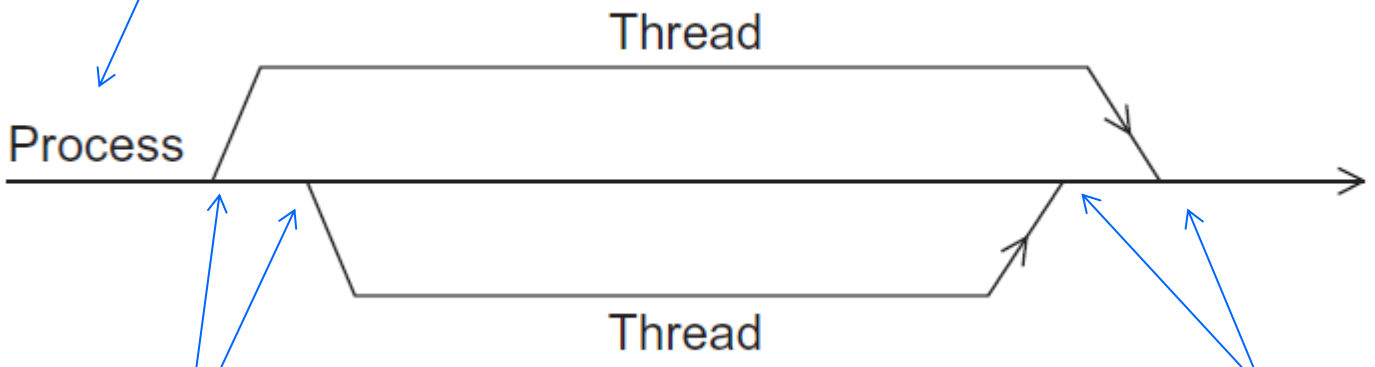
Multitasking



中山大學
SUN YAT-SEN UNIVERSITY

- Gives the illusion that a single processor system is running multiple programs simultaneously.
- Each process takes turns running. (**time slice**)
- After its time is up, it waits until it has a turn again.

A process and two threads



目录

- The von Neumann architecture
- Processes, multitasking, and threads
- Caching
  - 掌握基本工作机制
- Virtual memory
- Instruction-level parallelism
- Hardwear multithreading

Modifications to the von Neumann model

中山大學 SUN YAT-SEN UNIVERSITY

Basics of caching

- A collection of memory locations that can be accessed in less time than some other memory locations.

- A CPU cache is typically located on the same chip, or one that can be accessed much faster than ordinary memory.



Basics of caching

- Cache holds **copies** of recently used main memory data.
  - Not new address space; mostly **transparent**.
- Data is transferred in fixed-size **cache lines**.
  - Often 64B in current CPUs; the reason of spatial locality (discussed later)
- Cache has limited capacity → lines may be **evicted** (replacement).
- For parallel code, cache behavior strongly affects scalability.
  - Why?



Principle of locality

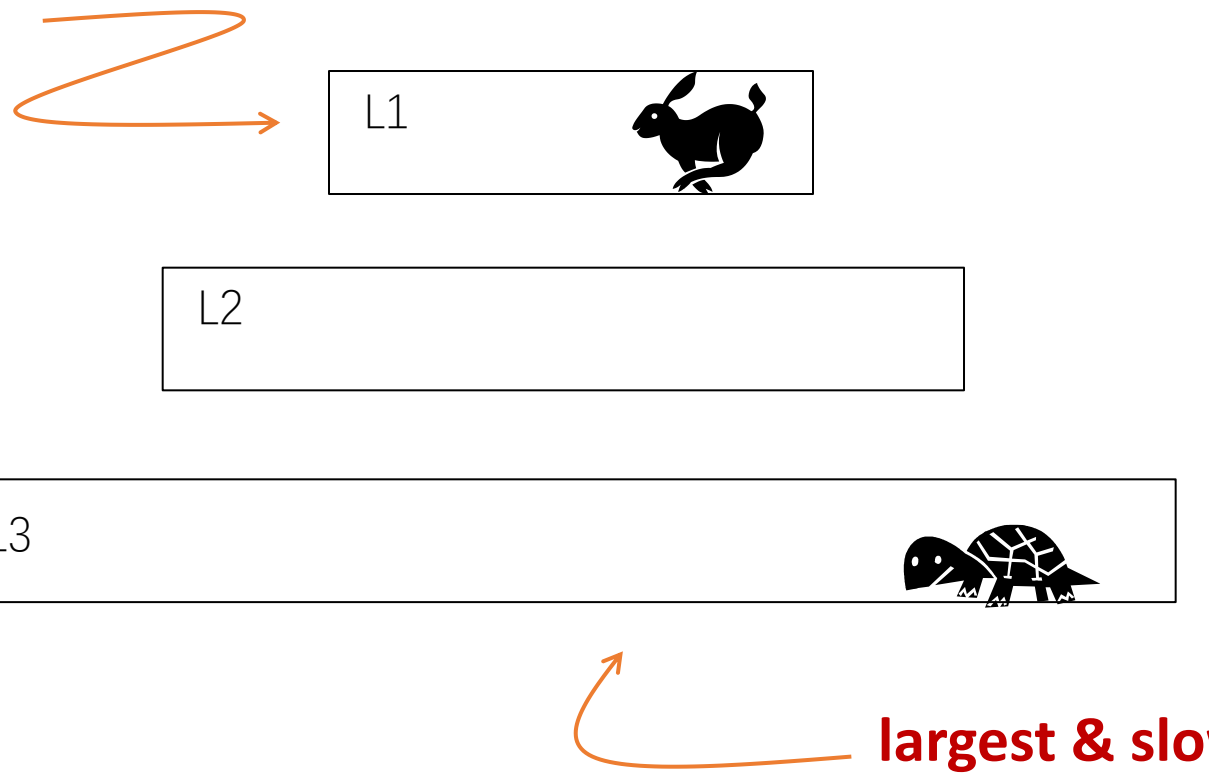


中山大學
SUN YAT-SEN UNIVERSITY

- **Spatial locality** – accessing a nearby location.
- **Temporal locality** – accessing in the near future.

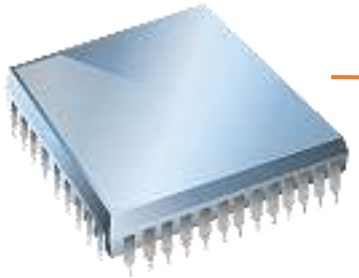
```
float z[1000];
…
sum = 0.0;
for (i = 0; i < 1000; i++)
   sum += z[i];
```

Levels of Cache



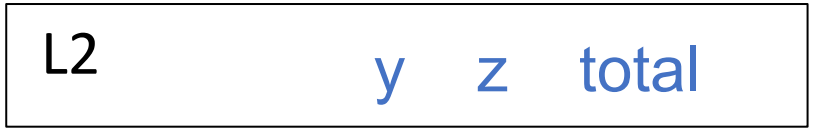
Cache hit

- When a cache is checked for information and the information is available, it's called a **cache hit** or just a **hit**



fetch x

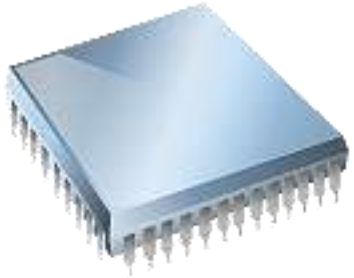




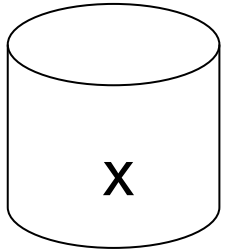


Cache miss

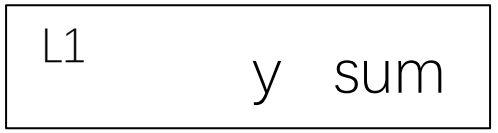
- If the information isn't available, it's called a **cache miss** or a **miss**.
- when a CPU attempts to access a data, it might encounter L1 miss and L2 hit

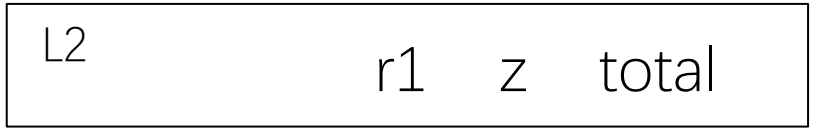






main memory







Coherence with memory



中山大學
SUN YAT-SEN UNIVERSITY

- When a CPU writes data to cache, the value in cache may be inconsistent with the value in main memory.

- **Write-through caches** handle this by updating the data in main memory at the time it is written to cache.

- **Write-back caches** mark data in the cache as dirty. When the cache line is replaced by a new cache line from memory, the dirty line is written to memory.

Coherence with memory~



中山大學
SUN YAT-SEN UNIVERSITY

- **Write-back caches** mark data in the cache as dirty. What if there is a "store miss"?
  - Write-allocate (a.k.a. fetch-on-write): The cache fetches the whole cache line containing [addr] from the next level, performs the write in the cache, and marks the line dirty
  - Write-around (a.k.a. no-write-allocate): The store bypasses the cache and goes directly to the next level
- When the cache line is replaced by a new cache line from memory, the dirty line is written to memory.
  - Actually, propagates down the hierarchy step by step

Cache mappings



中山大學
SUN YAT-SEN UNIVERSITY

- **Full associative** – a new line can be placed at any location in the cache.
- **Direct mapped** – each cache line has a unique location in the cache to which it will be assigned.
- ***n*-way set associative** – each cache line can be placed in one of *n* different locations in the cache.

n-way set associative

中山大學
SUN YAT-SEN UNIVERSITY

- When more than one line in memory can be mapped to several different locations in cache
- We also need to be able to decide which line should be replaced or evicted.
  - Least recently used principle



Example

Memory Index	Cache Location		
	Fully Assoc	Direct Mapped	2-way
0	0, 1, 2, or 3	0	0 or 1
1	0, 1, 2, or 3	1	2 or 3
2	0, 1, 2, or 3	2	0 or 1
3	0, 1, 2, or 3	3	2 or 3
4	0, 1, 2, or 3	0	0 or 1
5	0, 1, 2, or 3	1	2 or 3
6	0, 1, 2, or 3	2	0 or 1
7	0, 1, 2, or 3	3	2 or 3
8	0, 1, 2, or 3	0	0 or 1
9	0, 1, 2, or 3	1	2 or 3
10	0, 1, 2, or 3	2	0 or 1
11	0, 1, 2, or 3	3	2 or 3
12	0, 1, 2, or 3	0	0 or 1
13	0, 1, 2, or 3	1	2 or 3
14	0, 1, 2, or 3	2	0 or 1
15	0, 1, 2, or 3	3	2 or 3

Table 2.1: Assignments of a 16-line main memory to a 4-line cache

Caches and programs



中山大學
SUN YAT-SEN UNIVERSITY

```
double A[MAX][MAX], x[MAX], y[MAX];
. . .
/* Initialize A and x, assign y = 0 */
. . .
/* First pair of loops */
for (i = 0; i < MAX; i++)
   for (j = 0; j < MAX; j++)
      y[i] += A[i][j]*x[j];
. . .
/* Assign y = 0 */
. . .
/* Second pair of loops */
for (j = 0; j < MAX; j++)
   for (i = 0; i < MAX; i++)
      y[i] += A[i][j]*x[j];
```

Cache Line	Elements of A			
0	A[0][0]	A[0][1]	A[0][2]	A[0][3]
1	A[1][0]	A[1][1]	A[1][2]	A[1][3]
2	A[2][0]	A[2][1]	A[2][2]	A[2][3]
3	A[3][0]	A[3][1]	A[3][2]	A[3][3]

目录

- The von Neumann architecture
- Processes, multitasking, and threads
- Caching
- Virtual memory
  - 掌握基本工作机制
- Instruction-level parallelism
- Hardwear multithreading

Modifications to the von Neumann model

中山大學 SUN YAT-SEN UNIVERSITY

Virtual memory (1)



中山大學
SUN YAT-SEN UNIVERSITY

- A mechanism that makes each process “think” it owns a large, contiguous, private memory space (**virtual address space**), while the OS + hardware **map** that virtual space onto:
  - physical RAM (fast), and
  - optionally secondary storage (SSD/HDD, slow) as a backing store.

- Motivation
  - Memory management
  - Virtual memory functions as a cache for secondary storage.
  - Memory protection

Virtual memory (2)



- It exploits the principle of spatial and temporal locality.

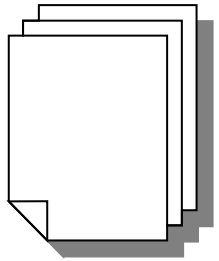
- It only keeps the active parts of running programs in main memory.

Virtual memory (3)

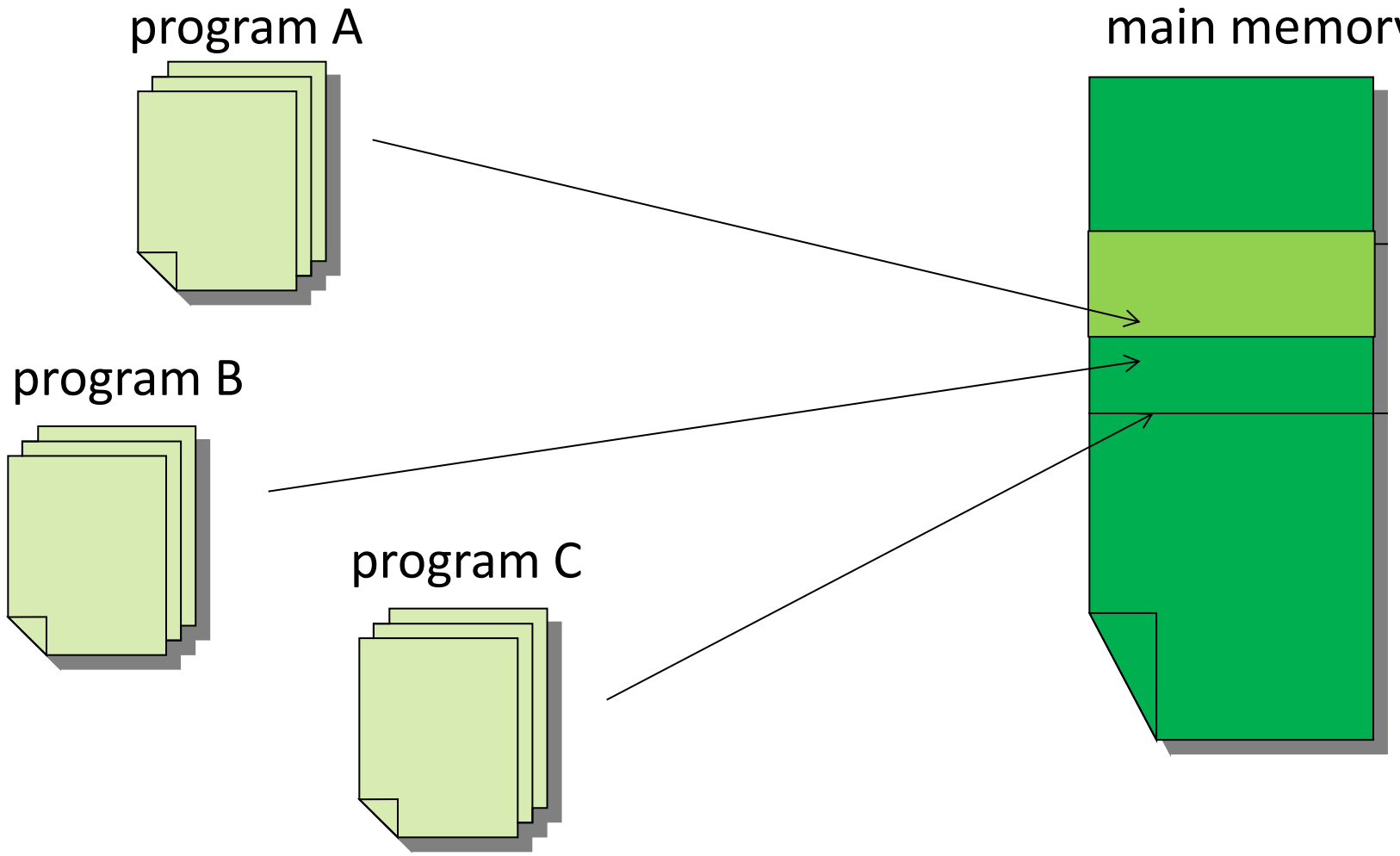


- **Swap space** - those parts that are idle are kept in a block of secondary storage.

- **Pages** – blocks of data and instructions.
  - Usually these are relatively large.
  - Most systems have a fixed page size that currently ranges from 4 to 16 kilobytes.



Virtual memory (4)



Virtual address

- The program uses virtual addresses, which imply virtual page numbers.
- When the program is running, a table is created that maps the virtual page numbers to physical addresses.
- A **page table** is used to translate the virtual address into a physical address.

Virtual address



中山大學
SUN YAT-SEN UNIVERSITY

Virtual Address									
Virtual Page Number					Byte Offset				
31	30	···	13	12	11	10	···	1	0
1	0	···	1	1	0	0	···	1	1

Table 2.2: Virtual Address Divided into Virtual Page Number and Byte Offset

Translation-lookaside buffer (TLB)



中山大學
SUN YAT-SEN UNIVERSITY

- Using a page table has the potential to significantly increase each program's overall run-time.
- A special address translation cache in the processor.
- It caches a small number of entries (typically 16–512) from the page table in very fast memory.
- **Page fault** – attempting to access a valid physical address for a page in the page table but the page is only stored on disk.

目录

- The von Neumann architecture
- Processes, multitasking, and threads
- Caching
- Virtual memory
- Instruction-level parallelism
  - 掌握基本工作机制
- Hardwear multithreading

Modifications to the von Neumann model

中山大學 SUN YAT-SEN UNIVERSITY

Instruction Level Parallelism (ILP)



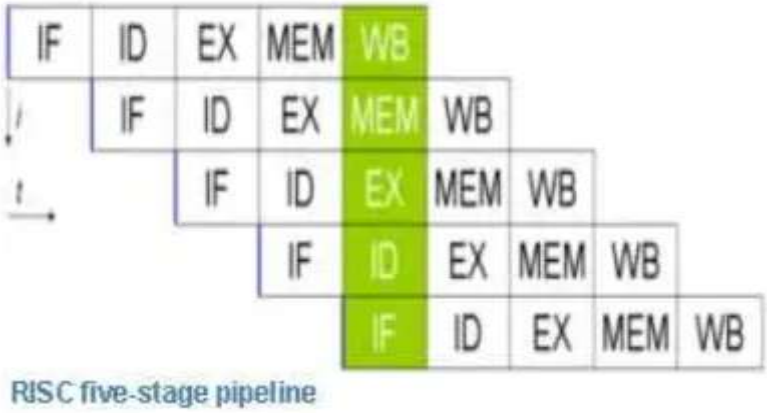
中山大學
SUN YAT-SEN UNIVERSITY

- Attempts to improve processor performance by having multiple processor components or **functional units** simultaneously executing instructions.

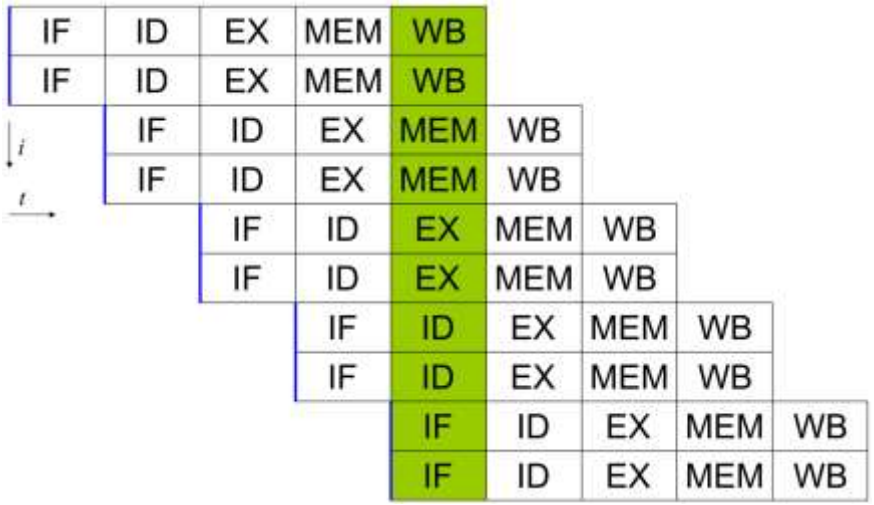
Instruction Level Parallelism (2)

中山大學 SUN YAT-SEN UNIVERSITY

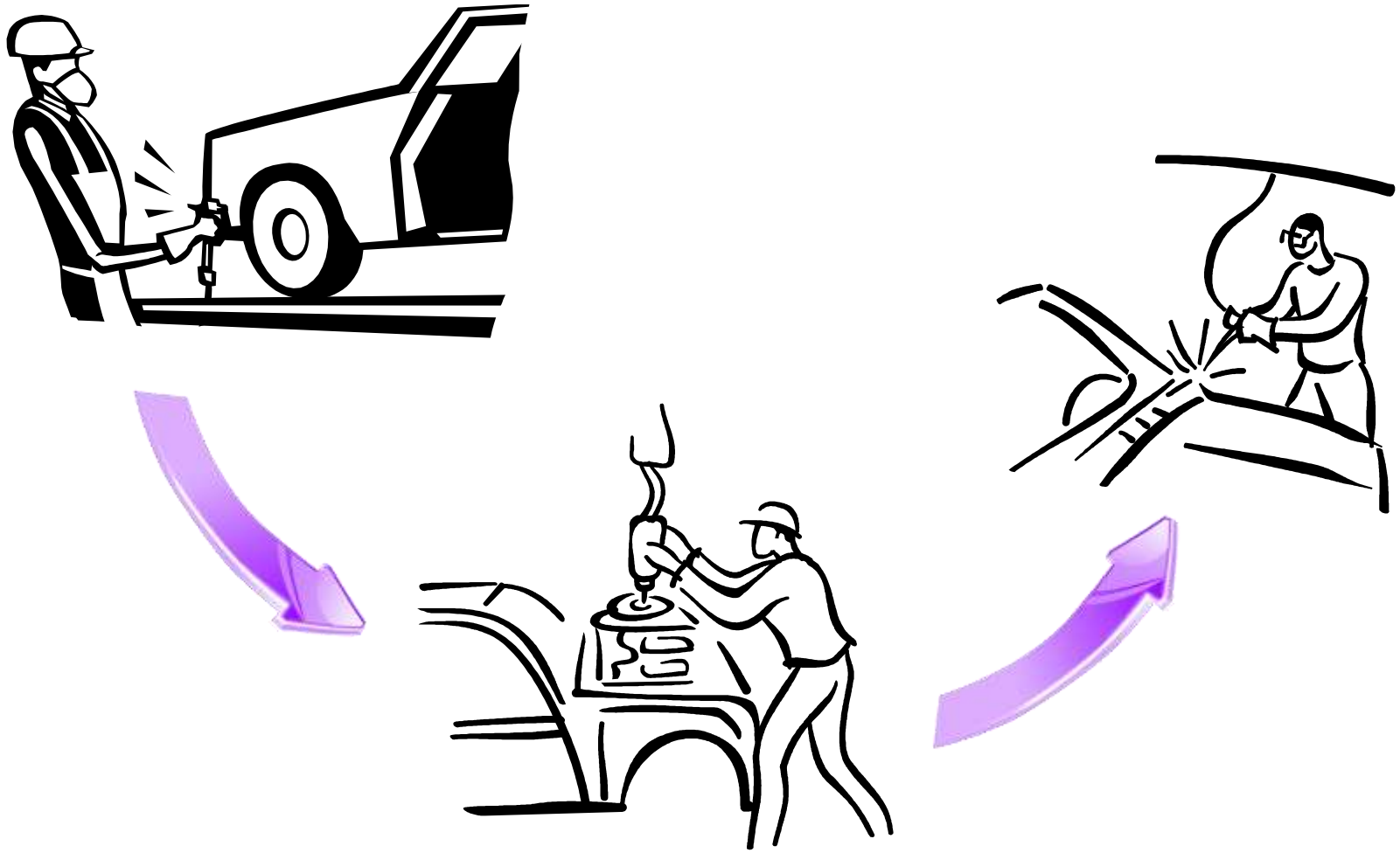
- **Pipelining** - functional units are arranged in stages.
- **Multiple issue** - multiple instructions can be simultaneously initiated.



RISC five-stage pipeline



Pipelining



Pipelining example (1)

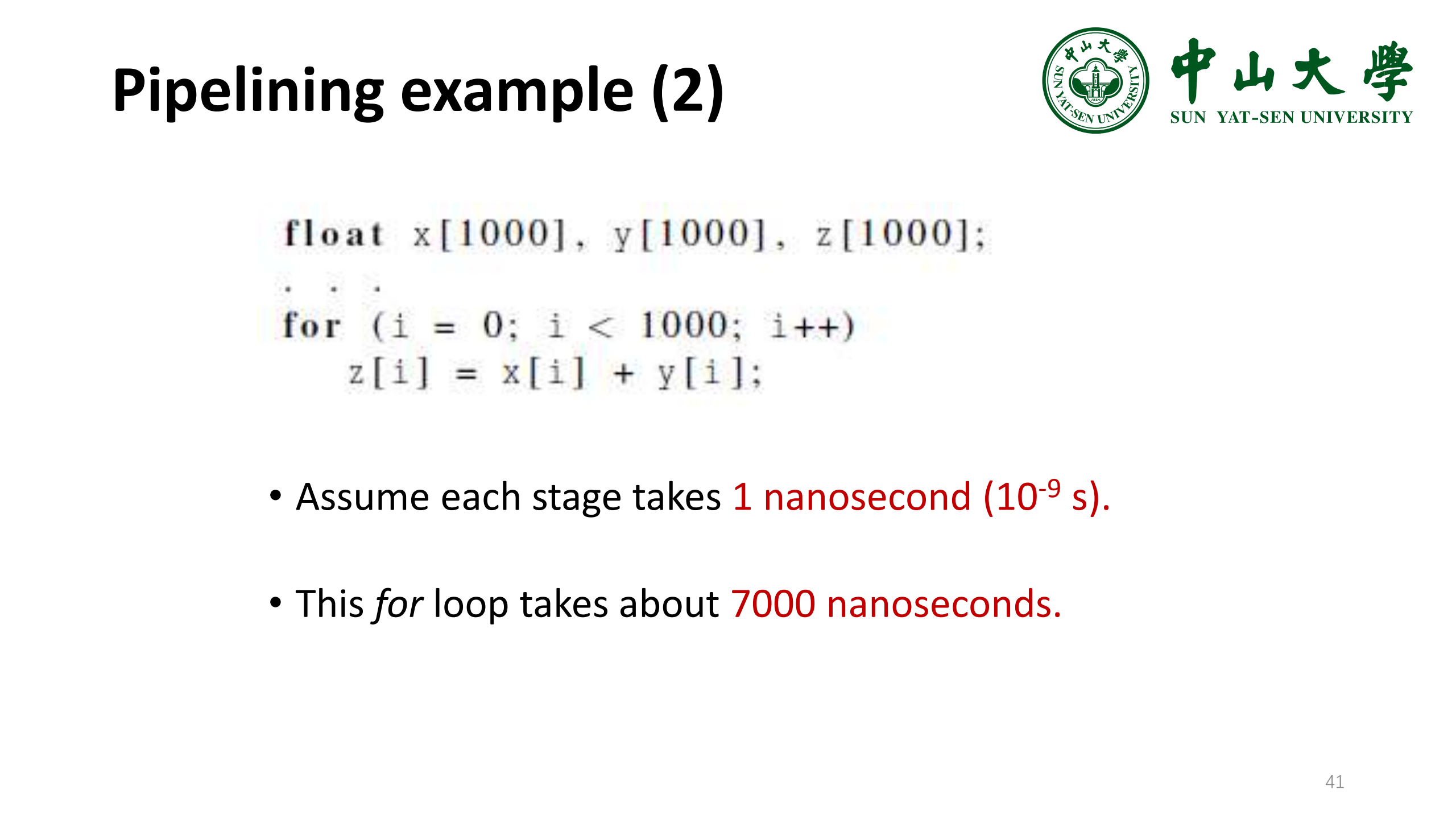


中山大學
SUN YAT-SEN UNIVERSITY

Time	Operation	Operand 1	Operand 2	Result
1	Fetch operands	9.87×10^4	6.54×10^3	
2	Compare exponents	9.87×10^4	6.54×10^3	
3	Shift one operand	9.87×10^4	0.654×10^4	
4	Add	9.87×10^4	0.654×10^4	10.524×10^4
5	Normalize result	9.87×10^4	0.654×10^4	1.0524×10^5
6	Round result	9.87×10^4	0.654×10^4	1.05×10^5
7	Store result	9.87×10^4	0.654×10^4	1.05×10^5

Add the floating point numbers
9.87×10^4 and 6.54×10^3

Pipelining example (2)




```
float x[1000], y[1000], z[1000];
. . .
for (i = 0; i < 1000; i++)
    z[i] = x[i] + y[i];
```

- Assume each stage takes 1 nanosecond (10^{-9} s).
- This *for* loop takes about 7000 nanoseconds.

Pipelining (3)



中山大學
SUN YAT-SEN UNIVERSITY

- Divide the floating point adder into 7 separate pieces of hardware or functional units.
- First unit fetches two operands, second unit compares exponents, etc.
- Output of one functional unit is input to the next.

Pipelining (4)



中山大學
SUN YAT-SEN UNIVERSITY

Time	Fetch	Compare	Shift	Add	Normalize	Round	Store
0	0						
1	1	0					
2	2	1	0				
3	3	2	1	0			
4	4	3	2	1	0		
5	5	4	3	2	1	0	
6	6	5	4	3	2	1	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
999	999	998	997	996	995	994	993
1000		999	998	997	996	995	994
1001			999	998	997	996	995
1002				999	998	997	996
1003					999	998	997
1004						999	998
1005							999

Table 2.3: Pipelined Addition.

Numbers in the table are subscripts of operands/results.

Pipelining (5)

- One floating point addition still takes 7 nanoseconds.
- But 1000 floating point additions now takes 1006 nanoseconds!



Multiple Issue (1)

中山大學 SUN YAT-SEN UNIVERSITY

- Multiple issue processors **replicate** functional units and try to simultaneously execute different instructions in a program.

```
for (i = 0; i < 1000; i++)
    z[i] = x[i] + y[i];
```



Multiple Issue (2)



- **static** multiple issue - functional units are scheduled at compile time.
  - Pro: have time to do more in-depth analysis of the program
  - Con: the run-time state is unknown to a compiler

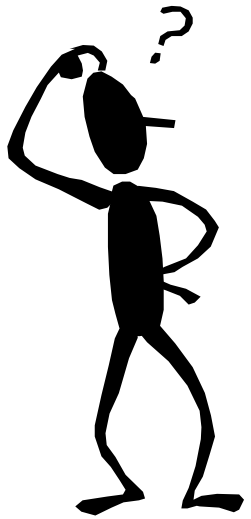
- **dynamic** multiple issue – functional units are scheduled at run-time.

superscalar

Speculation (1)

SUN YAT-SEN UNIVERSITY

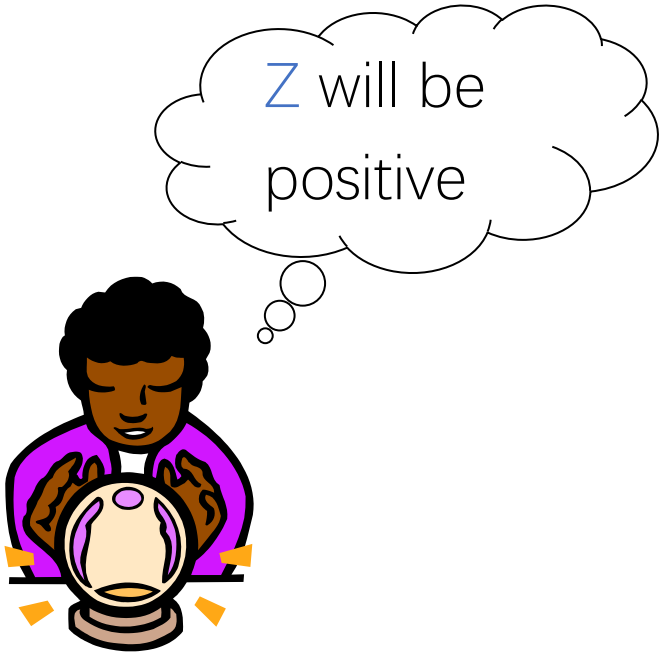
- In order to make use of multiple issue, the system must find instructions that can be executed simultaneously.



- In speculation, the compiler or the processor makes a **guess** about an instruction, and then executes the instruction on the basis of the guess.

Speculation (2)

```
z = x + y ;
i f ( z > 0)
     w = x ;
e l s e
     w = y ;
```



If the system speculates incorrectly,

it must go back and recalculate w = y.

目录

- The von Neumann architecture
- Processes, multitasking, and threads
- Caching
- Virtual memory
- Instruction-level parallelism
- Hardwear multithreading
  - 掌握基本工作机制

Modifications to the von Neumann model

中山大學 SUN YAT-SEN UNIVERSITY

Hardware multithreading (1)



中山大學
SUN YAT-SEN UNIVERSITY

- **Hardware multithreading** is when one physical CPU core is designed to keep track of multiple threads' architectural state (registers, program counter, etc.) so it can **switch between them in hardware**

- It provides a means for systems to continue doing useful work when the task being currently executed has stalled.
  - Ex., the current task has to wait for data to be loaded from memory.

Hardware multithreading (2)



中山大學
SUN YAT-SEN UNIVERSITY

- **Fine-grained** - the processor switches between threads after each instruction, skipping threads that are stalled.
  - **Pros**: potential to avoid wasted machine time due to stalls.
  - **Cons**: a thread that's ready to execute a long sequence of instructions may have to wait to execute every instruction.

Hardware multithreading (3)



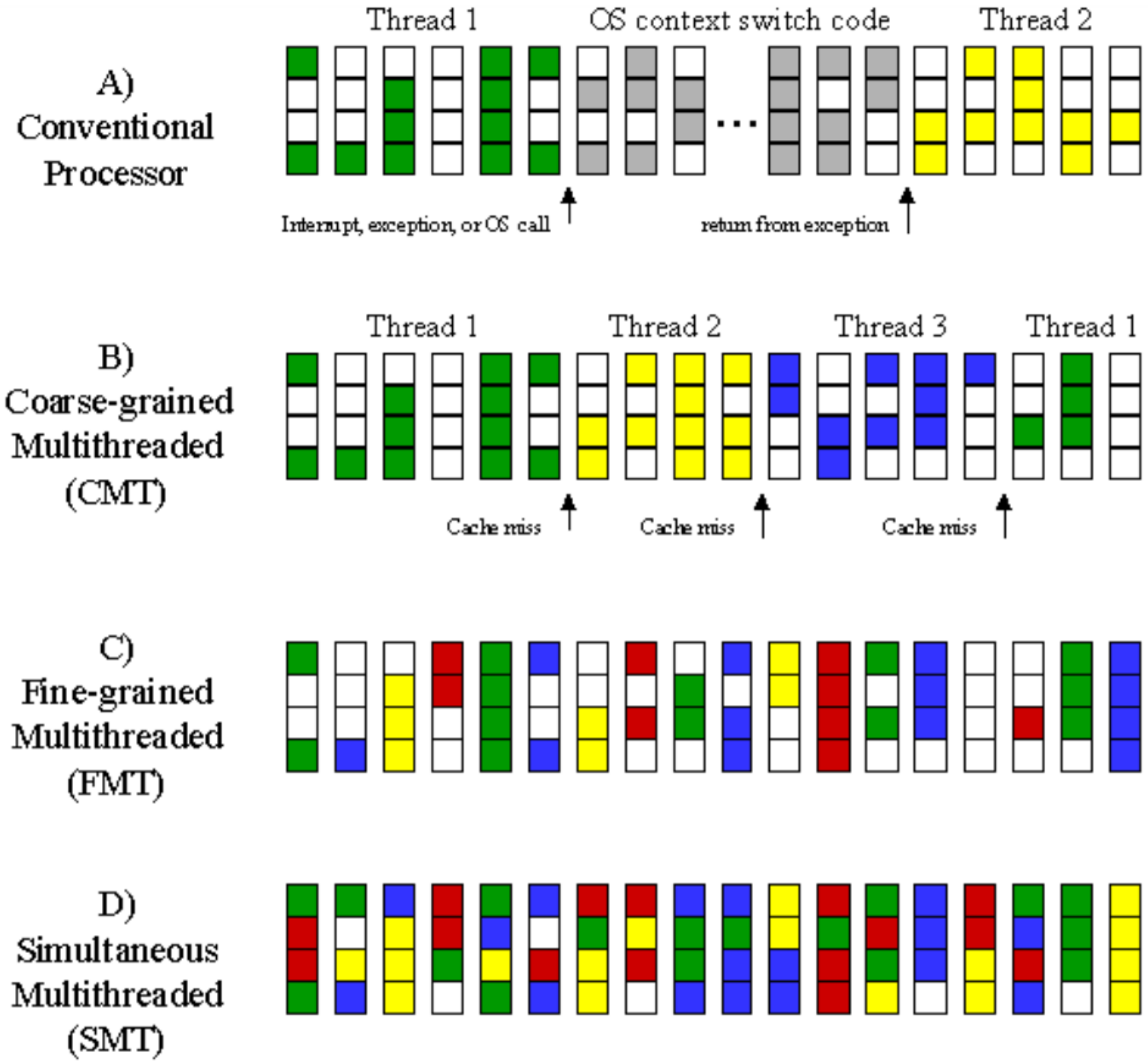
中山大學
SUN YAT-SEN UNIVERSITY

- **Coarse-grained** - only switches threads that are stalled waiting for a time-consuming operation to complete.
  - **Pros:** switching threads doesn't need to be nearly instantaneous.
  - **Cons:** the processor can be idled on shorter stalls, and thread switching will also cause delays.

Hardware multithreading (3)



- **Simultaneous multithreading (SMT)** - a variation on fine-grained multithreading.
- Allows multiple threads to make use of the multiple functional units.
- There aren't always good opportunities for simultaneous execution of different threads.


A)
Conventional
Processor
Thread 1
OS context switch code
Thread 2
Interrupt, exception, or OS call
return from exception
B)
Coarse-grained
Multithreaded
(CMT)
Thread 1
Thread 2
Thread 3
Thread 1
Cache miss
Cache miss
Cache miss
C)
Fine-grained
Multithreaded
(FMT)
D)
Simultaneous
Multithreaded
(SMT)

谢谢！

- Q&A

中山大學
SUN YAT-SEN UNIVERSITY

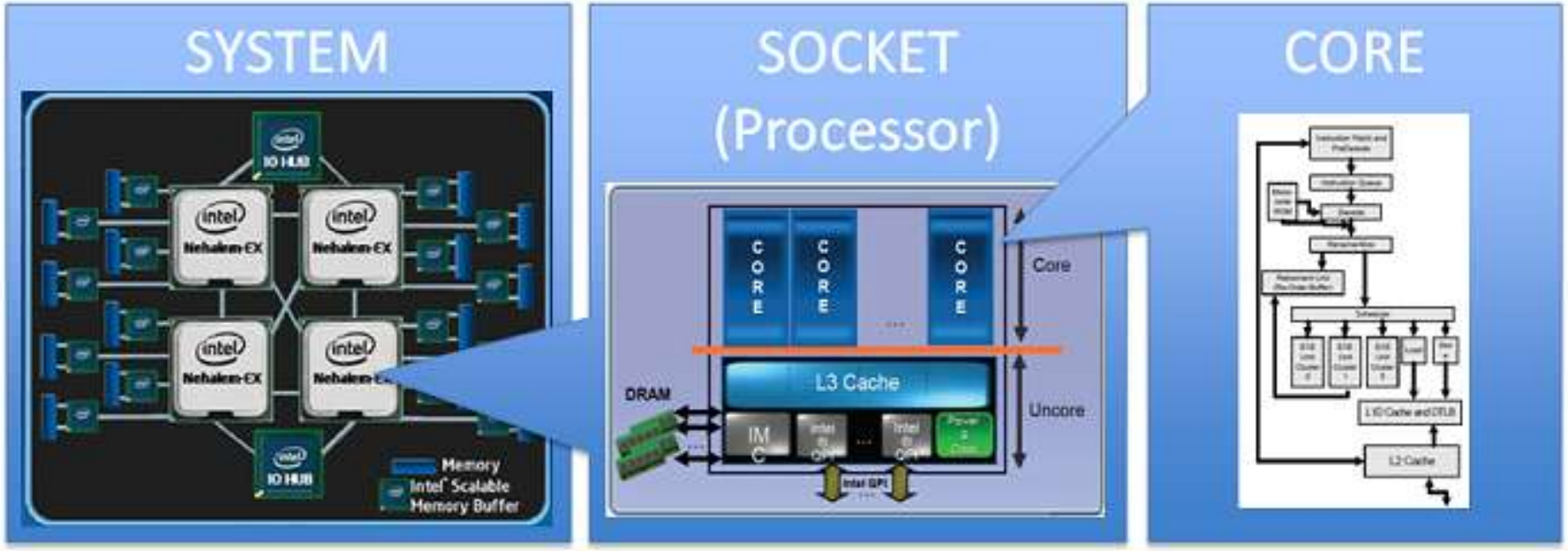
CPU vs. Core



- A **core** is usually the basic computation unit of the CPU
  - it can run a single program context
  - or multiple ones if it supports hardware threads such as hyperthreading on Intel CPUs

- A **CPU** may have one or more cores to perform tasks at a given time.
  - These tasks are usually software processes and threads that the OS schedules.
  - X = number cores * number of hardware threads per core.

CPU vs. Core



- A system contains (1 or many) CPUs
- A CPU contains (1 or many) cores
- Each core can execute (1 or many) threads depending on hyper-threading technology

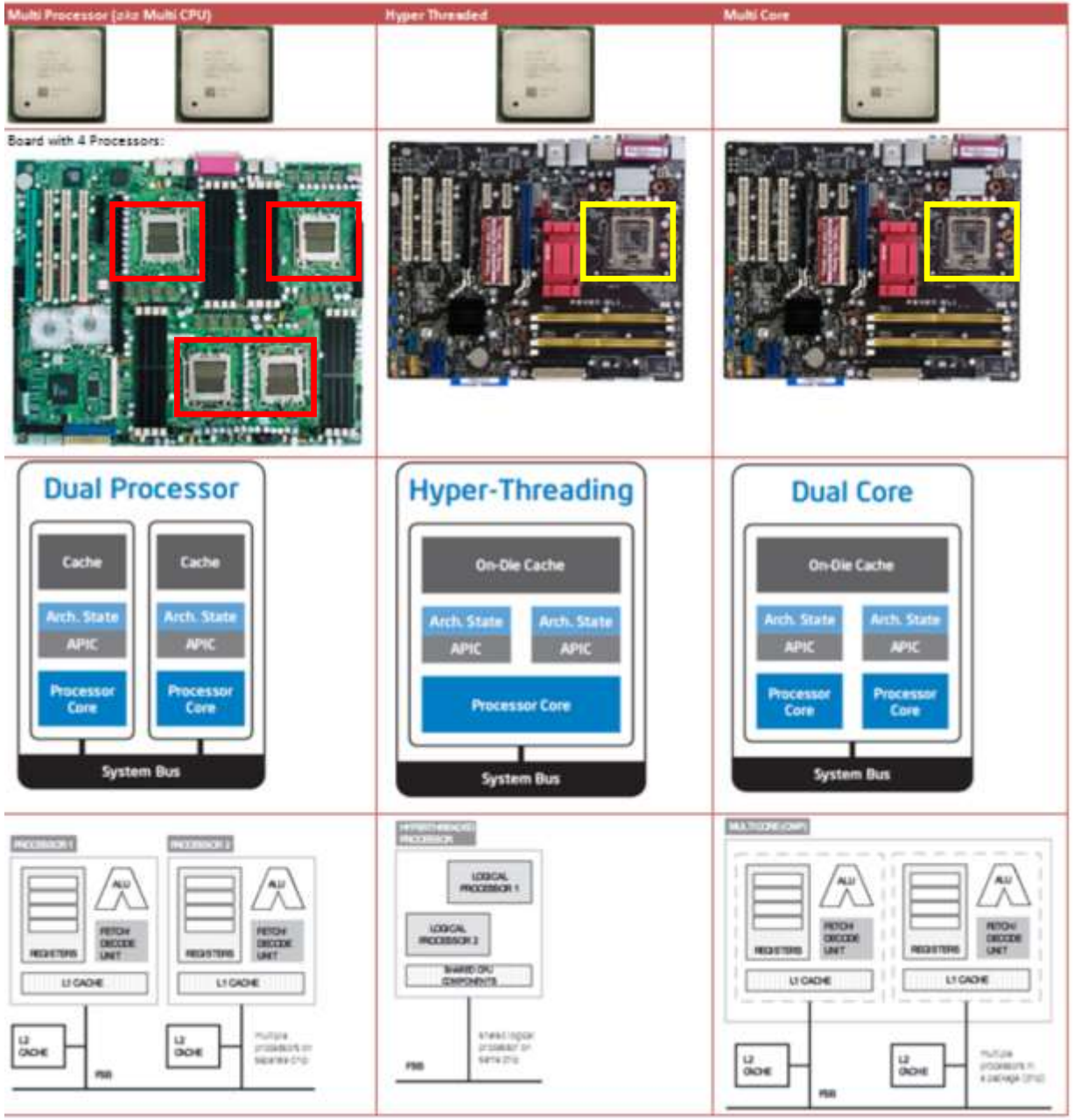
Processor vs. CPU

- A processor is a generic term used to describe any sort of CPU, regardless of cores.

- A multiprocessor system contains more than one CPU, allowing them to work in parallel.
  - This is called SMP, or Symmetric Multi-Processing.

- A multi*core* CPU has multiple cores on one CPU.

Multi-processor Single-core CPU Multi-Core CPU



Can a single **process** run in multiple cores?



- Yes, a single process can run multiple threads on different cores.

Can a single thread run in multiple cores?



中山大學
SUN YAT-SEN UNIVERSITY

- There is no such thing as a single thread running on multiple cores simultaneously.
  - Thread migration is possible

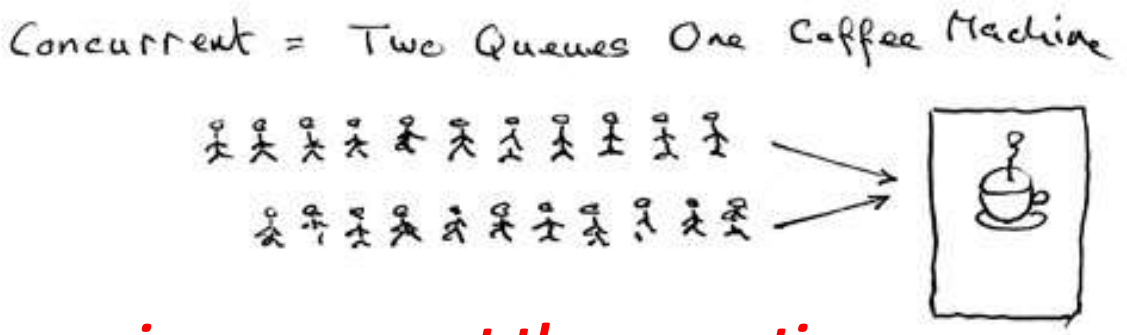
- It doesn't mean, however, that instructions from one thread cannot be executed in parallel.
  - There are mechanisms called instruction pipelining and out-of-order execution that allow it.

Concurrent vs. Parallel

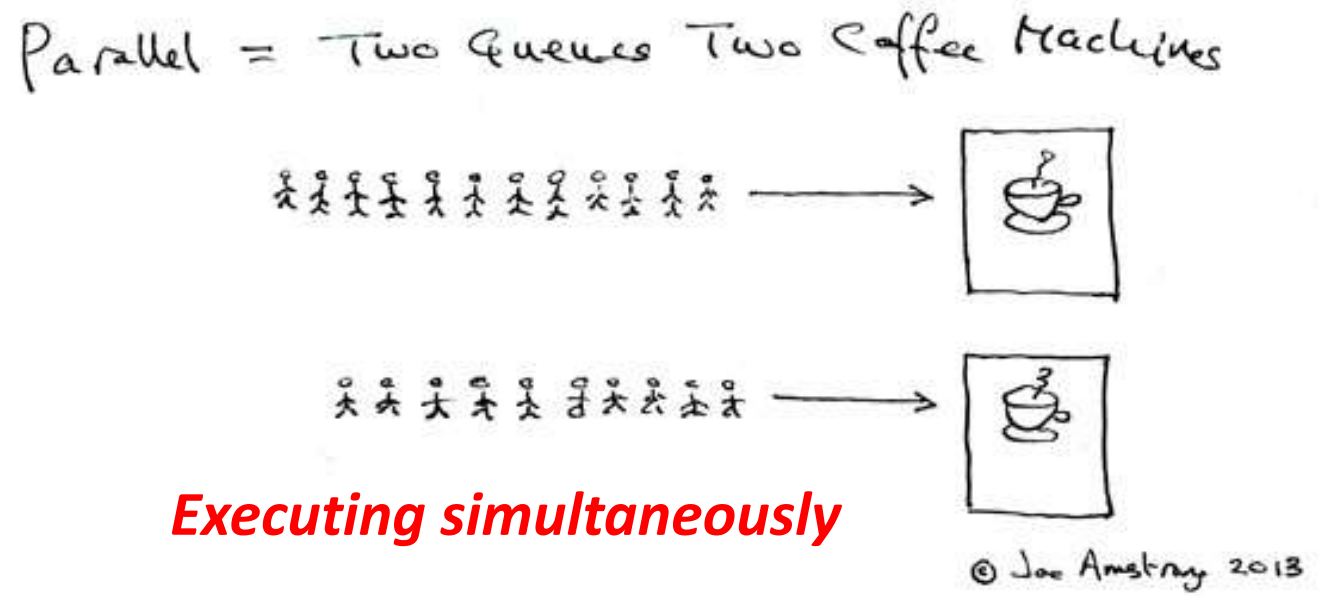
- A system is said to be **concurrent** if it can support two or more actions ***in progress at the same time***.

- A system is said to be **parallel** if it can support two or more actions executing ***simultaneously***.

- **Concurrent** = Two queues and one coffee machine.
- **Parallel** = Two queues and two coffee machines.



in progress at the same time.



Executing simultaneously

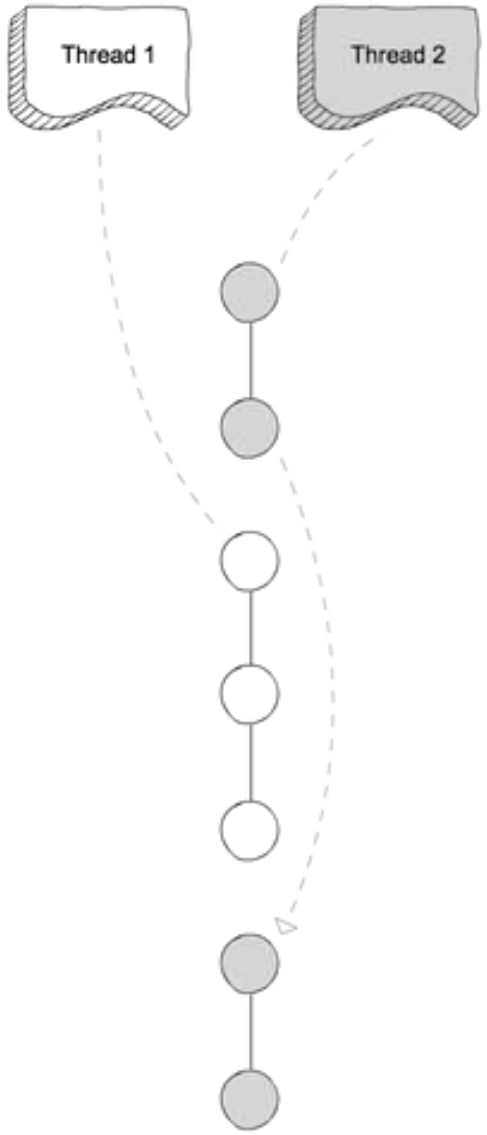
Parallel Computing vs. Concurrent Computing

- In parallel computing, execution occurs at the *same physical instant*
  - parallel computing is impossible on a (one-core) single processor

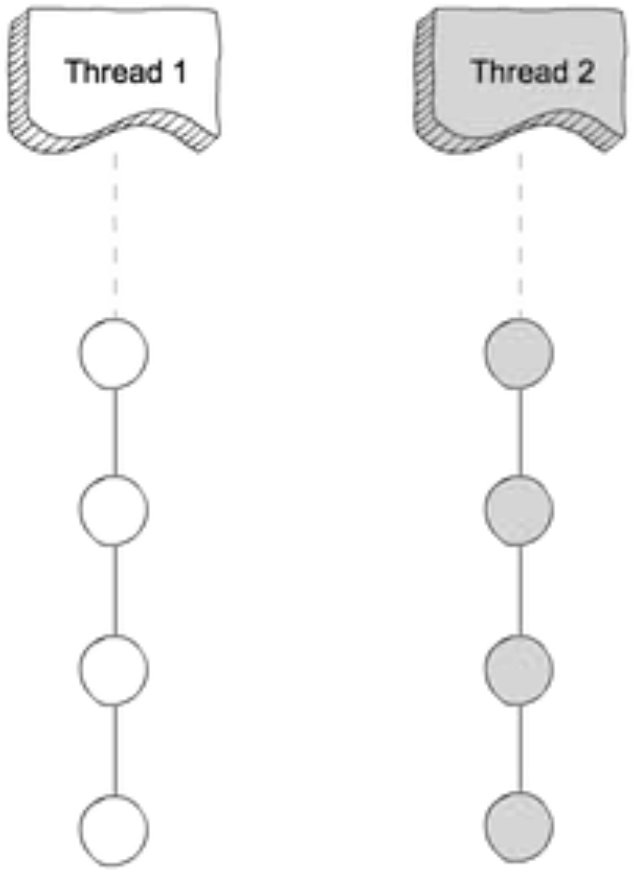
- concurrent computing consists of process lifetimes overlapping, but *execution need not happen at the same instant*
  - concurrent processes can be executed on one core by interleaving the execution steps of each process via time-sharing slices

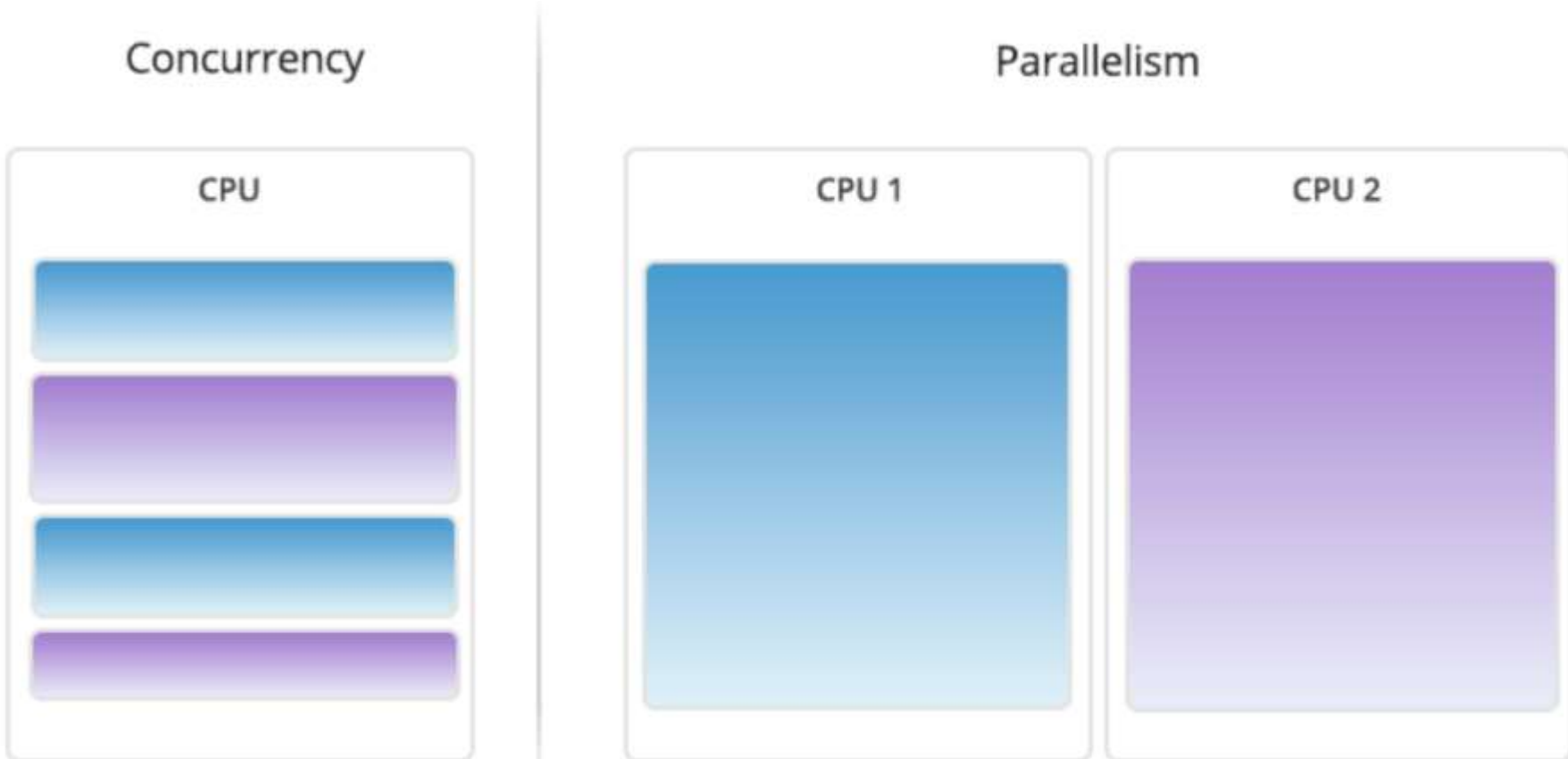
https://en.wikipedia.org/wiki/Parallel_computing

Concurrent Computing



Parallel Computing




Concurrency
CPU
Parallelism
CPU 1
CPU 2